SMS / TEXT SPAM CLASSIFIER

Using an Ensemble of Classical Machine Learning Models

Project Report

Submitted by:

DULAM ANVESH GOUD

REGD No.: 25BAI10595

AI/ML Course Project :2025-26

1. Introduction and Problem Statement

Digital communication has become the backbone of personal and professional life, yet it is heavily plagued by unsolicited and fraudulent messages known collectively as spam. Spam messages range from harmless promotional texts to highly deceptive phishing attempts designed to steal money or sensitive information from unsuspecting recipients.

The objective of this project is to build a terminal-based, interactive spam classification system capable of deciding in real time whether a given SMS or text message is spam or legitimate (ham). The system is designed to be practical, explainable, and continuously improvable through human feedback.

1.1 Why This Problem Matters

Spam messages are not a minor inconvenience — they represent a serious security risk. Common attack vectors include:

- Phishing links impersonating banks, payment wallets, or government services
- Fake prize or lottery announcements designed to extract OTPs or bank details
- Fraudulent job or loan offers targeting financially vulnerable individuals
- Fake delivery or customs-fee scams disguised as legitimate courier alerts

In India alone, millions of users receive multiple spam messages daily via WhatsApp, SMS, and email. Automated, accurate, and real-time filtering is therefore both practically valuable and technically challenging, particularly because spam patterns constantly evolve.

1.2 Scope of the Project

The project focuses on binary text classification (spam vs. ham) using NLP pre-processing and five classical machine learning algorithms combined through majority voting. The solution is delivered as a fully interactive terminal-based Python application that trains itself on startup, accepts live messages, explains its reasoning, collects user feedback, and exports results to CSV.

2. Approach and Methodology

2.1 High-Level Architecture

The system is composed of four major layers: a curated and synthetic training dataset, a TF-IDF feature extraction pipeline, an ensemble of five independently trained classifiers, and a majority-voting prediction layer with confidence display and a user feedback loop. The data flows as follows:

```
Raw Message  -->  TF-IDF Vectorizer  -->  [NB | LR | DT | RF | SVM]  -->
          Majority Vote  -->  Verdict + Confidence + Keywords
```

2.2 Dataset Construction

A clean, balanced dataset is fundamental to classifier quality. The dataset was assembled in three stages:

Stage 1 - Manually Curated Examples

Approximately 77 spam and 77 ham messages were written by hand. The spam messages mimic real-world patterns such as prize notifications, phishing alerts, loan offers, and OTP scams, with both formal English and colloquial Indian-English phrasing. The ham examples cover everyday student and personal communication such as class reminders, canteen updates, and project coordination messages.

Stage 2 - Synthetic Data Augmentation

A template-based generator was written to produce 350 additional spam and 350 additional ham messages by filling parameterised sentence templates with slot values covering amounts, service names, time slots, room numbers, and more. This ensured diversity without overfitting to any single phrasing style while keeping the class distribution balanced.

Stage 3 - Dynamic Feedback Integration

A feedback mechanism allows users to flag incorrect predictions during a session. Each correction is appended to a persistent feedback.csv file and incorporated into the training dataset on the next run, enabling continuous learning from real-world corrections without redeploying the code.

Dataset Component	Approximate Count
Manually curated spam	~77 messages
Manually curated ham	~77 messages
Synthetic spam (template-generated)	350 messages
Synthetic ham (template-generated)	350 messages
User feedback (dynamic)	Variable (grows over use)

Dataset Component	Approximate Count
Total (base)	~854 messages

2.3 Feature Engineering - TF-IDF

Text messages were converted into numerical feature vectors using Term Frequency-Inverse Document Frequency (TF-IDF). The vectorizer was configured with a vocabulary cap of 500 features to control dimensionality, English stop words removed to focus on content-bearing terms, and all tokens lowercased for case-insensitive matching.

TF-IDF was chosen over simpler Bag-of-Words because it naturally down-weights common words that appear across all messages and up-weights terms that are distinctive to a particular class. Words like 'prize', 'click', 'free', and 'urgent' thus receive high TF-IDF scores in spam messages, making them strong discriminating features.

2.4 Classifier Ensemble

Five diverse classifiers were trained so that their individual weaknesses could be compensated by ensemble agreement:

Algorithm	Rationale	Confidence Source
Multinomial Naive Bayes	Fast; strong prior for sparse NLP features	predict_proba()
Logistic Regression	Linear, interpretable, well-regularised	predict_proba()
Decision Tree	Non-linear splits; captures keyword rules	predict_proba()
Random Forest (100 trees)	Bagging reduces single-tree overfitting	predict_proba()
SVM (Calibrated)	Maximum-margin; excellent on TF-IDF vectors	Sigmoid of decision_function()

2.5 Majority Voting Strategy

The final verdict for any message is determined by a simple majority vote across the five models. If three or more models predict SPAM, the overall verdict is SPAM; otherwise, the message is classified as HAM. This democratic strategy reduces the risk of a single poorly-fitting model dominating the output.

Confidence is reported as the fraction of models agreeing with the majority verdict, expressed as a percentage. A 5-0 vote yields 100% confidence; a 3-2 vote yields 60%, correctly signalling ambiguity. The vote count itself (e.g., '3/5 models say SPAM') is also displayed so users can interpret close calls accurately.

3. Key Design Decisions

3.1 Classical ML over Deep Learning

A deliberate choice was made to use classical machine learning rather than transformer-based models such as BERT. Classical models train in seconds with no GPU required, the dataset size of roughly 850 samples is too small to fine-tune deep models reliably, inference is instantaneous making the app feel responsive, and models are fully explainable via TF-IDF feature importance.

3.2 Ensemble over Single Best Model

Early experiments showed that no single model dominated across all message types. Naive Bayes performed well on short keyword-heavy messages but struggled with nuanced ham. The Decision Tree had higher variance. By combining all five models through voting, the ensemble achieved more stable and reliable results across diverse message styles.

3.3 Model Persistence with Joblib

Trained models and the fitted TF-IDF vectorizer are saved to disk using joblib after each training run. On subsequent launches the app loads persisted models instead of retraining from scratch, reducing startup time significantly. Retraining is triggered only when new feedback examples are available or the user explicitly requests it via the Force Retrain option.

3.4 Full Re-fit on Complete Data After Evaluation

The training procedure follows a two-pass strategy. In the first pass, models are fitted on a stratified 80/20 train-test split to measure accuracy and F1 scores for display purposes. In the second pass, each model is re-fitted on the entire dataset. This maximises the amount of data the live prediction model has seen, a standard practice when deploying a final model after evaluation.

3.5 Keyword Explainability

For every message classified as SPAM, the top-5 TF-IDF terms from the message are surfaced as 'spam signals'. This was implemented by transforming the input message through the vectorizer and selecting features with the highest TF-IDF scores. This explainability feature helps users understand why a message was flagged, builds trust in the system, and serves as an educational tool about how spam language works.

3.6 Session State and Summary Analytics

All predictions within a session are stored in memory in a session history list. A full summary is printed when the user exits, showing total messages classified, spam/ham counts, spam percentage, and the top-3 recurring spam keywords across the session. This transforms the app from a single-shot classifier into a mini analytics dashboard.

4. Challenges Faced

4.1 Class Imbalance in Initial Dataset

The initial manual dataset had a slight imbalance in message styles - spam examples tended to be longer and more varied, while ham examples were shorter and more repetitive. This caused early models to be biased toward detecting spam while misclassifying short ambiguous ham messages. The issue was addressed by adding more diverse ham templates in the synthetic stage and ensuring stratified train-test splitting.

4.2 SVM Probability Calibration

LinearSVC is among the best-performing linear classifiers on sparse TF-IDF matrices but does not support predict_proba(). Using raw decision_function() scores directly as confidence values produced poorly scaled numbers. The solution was to wrap LinearSVC in CalibratedClassifierCV, which applies Platt scaling via logistic regression over the SVM's scores, yielding proper probabilities comparable to those of the other models.

4.3 Vocabulary Generalisation to Indian-English Spam

A classifier trained on English templates initially failed to detect Indian-language-influenced spam mixing Hindi words with English such as 'yaar', 'paytm', 'aadhaar', and 'rupees'. This required adding a separate curated block of 50+ colloquial Indian-English spam and ham examples to the training set, ensuring the vocabulary captured common Hinglish spam patterns that are highly relevant to the target user base.

4.4 Honest Confidence Display for Close Votes

In early versions, a 3-2 SPAM vote would report 60% confidence, which felt misleadingly low. After reflection the confidence formula was kept mathematically honest - a 3-2 vote genuinely does represent a close call - and the UI was adjusted to show the exact vote count alongside the percentage so users could interpret the nuance correctly.

4.5 Feedback Loop and Stale Model Files

When feedback examples are available in feedback.csv, the app needs to retrain even if saved model files exist on disk. An edge case arose where the app loaded stale saved models and ignored new feedback. This was resolved by checking whether feedback was loaded at startup and, if so, forcing a fresh training run before returning the models, ensuring the live prediction model always incorporates the latest corrections.

5. Results and Performance

5.1 Per-Model Evaluation Metrics

Each model is evaluated on a stratified 20% held-out test set. Representative performance figures observed during testing are shown below. Exact numbers vary slightly between runs due to random shuffling in dataset construction.

Model	Accuracy (%)	F1-Score (%)
Naive Bayes	94 - 96	93 - 95
Logistic Regression	96 - 98	96 - 98
Decision Tree	91 - 94	90 - 93
Random Forest (100 trees)	96 - 99	95 - 98
SVM (Calibrated)	96 - 99	95 - 98
Ensemble (Majority Voting)	97 - 99	97 - 98

5.2 Sample Test Results

The built-in quick sample test (Menu Option 2) evaluates six pre-loaded messages - three spam and three ham - and prints the verdict, confidence, and accuracy against ground truth. In all test runs conducted, the system correctly classified all six samples at high confidence typically above 80%, demonstrating that the ensemble reliably captures both obvious keyword-heavy spam and more subtle legitimate messages.

5.3 Qualitative Observations

- Short, keyword-rich spam messages such as 'Win FREE iPhone now!' are detected with near-100% confidence by all five models
- Borderline messages such as professionally worded bank alerts sometimes split votes 3-2, correctly signalling ambiguity to the user
- Ham messages containing the word 'free' in a non-spam context, such as 'free period in college today', sometimes receive lower confidence scores - a known limitation of bag-of-words approaches
- Indian-English spam patterns involving rupees, paytm, aadhaar, and OTP are reliably detected after the Hinglish dataset was incorporated
- Batch mode and CSV export function correctly across all tested .txt and .csv file inputs

6. What I Learned

6.1 End-to-End ML Pipeline Design

Building the full pipeline from scratch - dataset curation, feature engineering, training, evaluation, prediction, explainability, feedback loop, and persistence - gave a hands-on understanding of what a production machine learning workflow looks like beyond textbook examples. Each layer exposed its own set of practical decisions and trade-offs.

6.2 Data Quality over Data Quantity

The most impactful improvement to model performance was not adding more data but adding more diverse and representative data. Adding 50 Hinglish spam examples improved Indian-language detection more than adding 200 additional English template variations. This reinforced the principle that coverage and quality of training examples matters more than raw count.

6.3 Probability Calibration is Non-trivial

Working with LinearSVC taught me that not all models produce calibrated probabilities. A raw SVM decision score does not carry the same meaning as a predict_proba() output. Applying CalibratedClassifierCV transformed those raw scores into comparable and interpretable probabilities - a step often skipped in tutorials but critical in practice when comparing confidence across models.

6.4 Ensemble Methods Reduce Fragility

No single classifier was perfect. By combining five diverse models, each with different inductive biases, the ensemble was more robust to individual model errors. When one model misclassified a borderline message the other four typically outvoted it. This was a vivid practical demonstration of why ensemble methods dominate real-world machine learning applications.

6.5 UX in ML Tools Matters

Designing the terminal interface revealed how important user experience is even in non-graphical tools. Adding colour output, a confidence bar, vote counts, spam keyword highlights, session history, batch mode, CSV export, and the feedback loop transformed a simple classify() function into a tool someone would actually want to use. Good UX builds trust in the underlying model.

6.6 Human-in-the-Loop Systems

The feedback mechanism demonstrated the concept of a human-in-the-loop system in practice. When a user corrects a misprediction that information is stored and used to update the model on the next training run. This mirrors how real-world production ML systems improve over time with user corrections and highlighted the ethical importance of giving users agency to dispute automated decisions.

7. Potential Future Enhancements

The following directions were identified as valuable extensions beyond the current implementation:

- Replace TF-IDF with pre-trained word embeddings such as FastText or sentence-transformers for better semantic understanding
- Add a web interface via Flask or Streamlit to make the tool accessible to non-technical users
- Integrate online learning so the model updates incrementally in real time without a full retrain cycle
- Add multi-class classification to distinguish between spam categories such as phishing, promotional, and financial scam
- Deploy as a REST API callable from SMS gateways or mobile applications
- Introduce explainability visualisations using LIME or SHAP to show per-feature contributions for each prediction
- Add confidence thresholds and a 'uncertain' category so borderline messages can be flagged for human review

8. Conclusion

This project successfully delivered a terminal-based, real-time spam classification system built on a five-model ensemble of classical machine learning algorithms. The system addresses a genuine and growing problem - the proliferation of fraudulent and phishing messages - with an approach that is accurate, interpretable, and continuously improvable through user feedback.

By combining TF-IDF feature extraction with majority voting over Naive Bayes, Logistic Regression, Decision Tree, Random Forest, and Calibrated SVM classifiers, the system achieves ensemble accuracy in the range of 97 to 99 percent on held-out test data. Beyond accuracy, the project implemented practical features - keyword explainability, user feedback integration, model persistence, batch processing, and session analytics - that bridge the gap between a classroom model and a tool that could realistically be deployed in a real environment.

The most important lesson from this project is that machine learning engineering is not only about choosing the right algorithm. It involves thoughtful data design, careful evaluation, deliberate UX decisions, and building feedback mechanisms that allow the system to grow smarter with use. This project has been a thorough exercise in all of those dimensions.

-- End of Report --